

Introduction to Neural Networks

Autoencoders

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

April 23, 2026©



Contents

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders
- 4 Denoising autoencoders
- 5 Regularized autoencoders
- 6 Variational autoencoders

Roadmap

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders
- 4 Denoising autoencoders
- 5 Regularized autoencoders
- 6 Variational autoencoders

Motivation



Figure: What define the images?

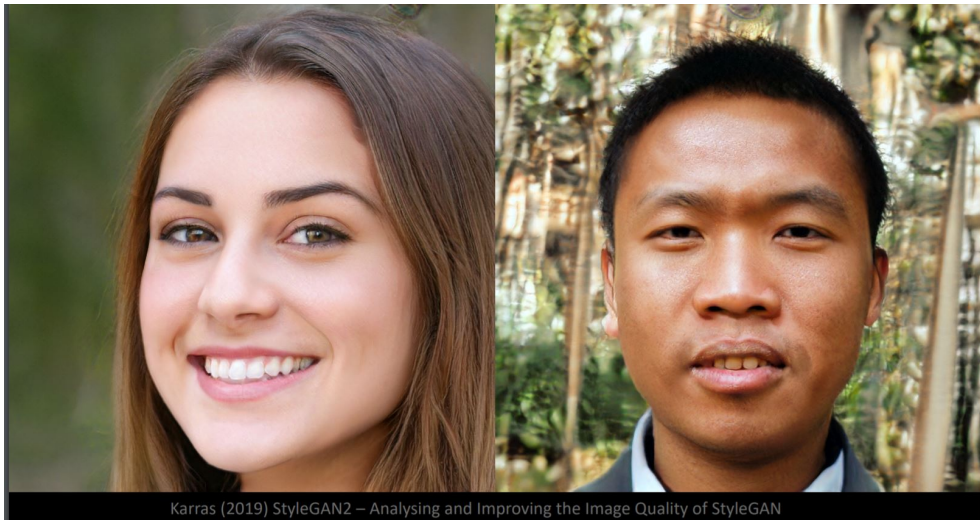
Motivation



Figure: What define the images?

- Examples can be defined by high level features.
- We will call to those features **code**

Applications of generative modeling

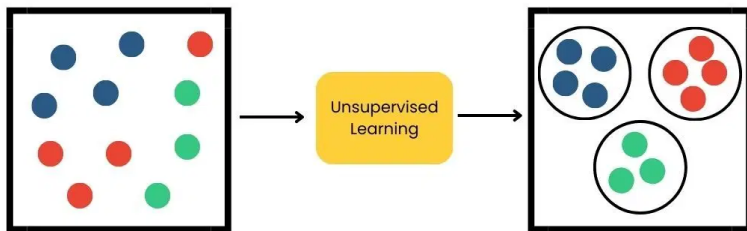


Roadmap

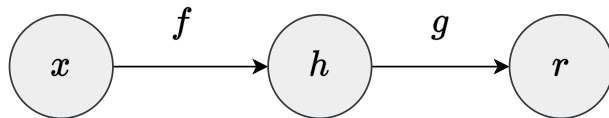
- 1 Motivation
- 2 Autoencoders**
- 3 Convolutional autoencoders
- 4 Denoising autoencoders
- 5 Regularized autoencoders
- 6 Variational autoencoders

Unsupervised learning

- Goal: learn structure from **unlabeled** data.



- Benefits: denoising, compression, pretraining, and generative modeling.



- An autoencoder is a neural network that is trained to try to copy its input to its output

$$h = f(x) \tag{1}$$

, h is an embedding that describes the input, where f is an encoder function.

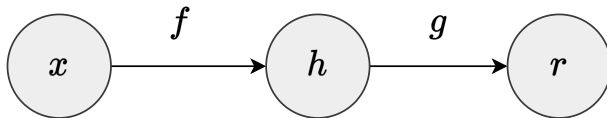
$$r = g(h) \tag{2}$$

is a reconstruction of the input, where g is a decoder function. We expect that:

$$r \simeq x$$

Autoencoders

Notation



- An autoencoder with parameters is written as:

$$r = g_{\phi}(f_{\theta}(x)) \quad (3)$$

where θ are the encoder parameters and ϕ are the decoder parameters.

- Condensed representation:

$$r = \Phi_{\theta, \phi}(x) \quad (4)$$

The learning process minimizes the reconstruction loss:

$$\min \mathcal{L}(x, g(f(x))) \quad (5)$$

e.g.

$$\mathcal{L}(x, r) = \sum_i (x_i - r_i)^2 \quad (6)$$

Undercomplete autoencoders

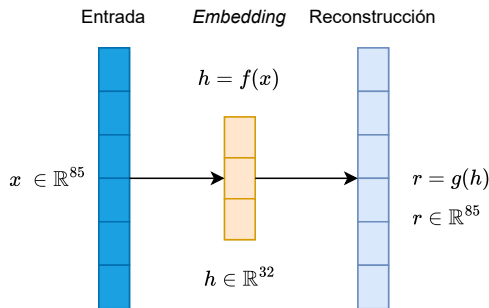
Bottleneck

- The embedding is smaller than input.
- Learning useful features in the bottleneck:

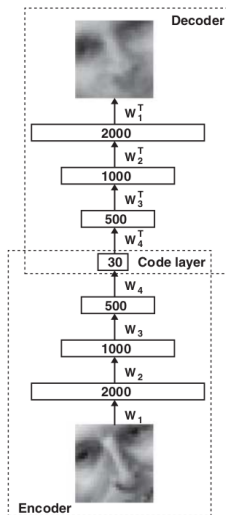
$$|x| = |r|$$

$$|h| \ll |x|$$

- The latent space, $\mathcal{H}$, captures the most relevant structure of $\mathcal{X}$.



Dimensionality reduction ¹

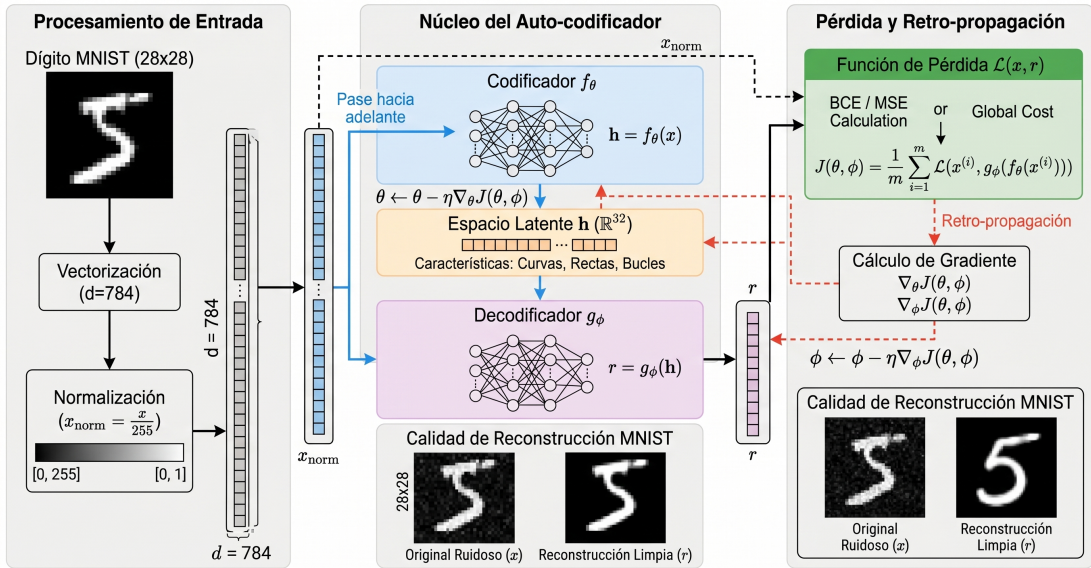


- **Cross-Entropy Error for Logistic Output Units** (Binary/Continuous data in range $[0, 1]$).

$$-\sum_i p_i \log \hat{p}_i - \sum_i (1 - p_i) \log(1 - \hat{p}_i)$$

- This function was used for the synthetic "curves" data and the MNIST handwritten digit data, where pixel intensities are between 0 and 1.

¹Hinton and Ruslan "Reducing the dimensionality of data with neural networks." science 313:5786 (2006): 504-507



Comparison against PCA

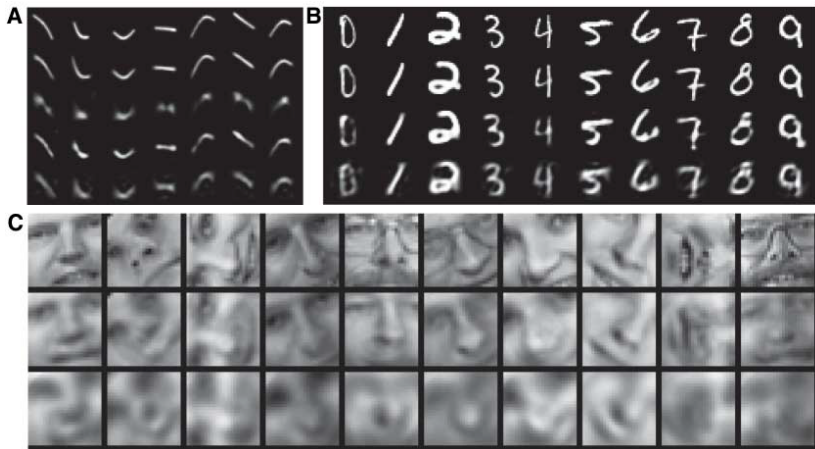


Figure: Rows: Original, autoencoder, logistic PCA 6, log PCA 18, standard PCA.

Comparison against PCA

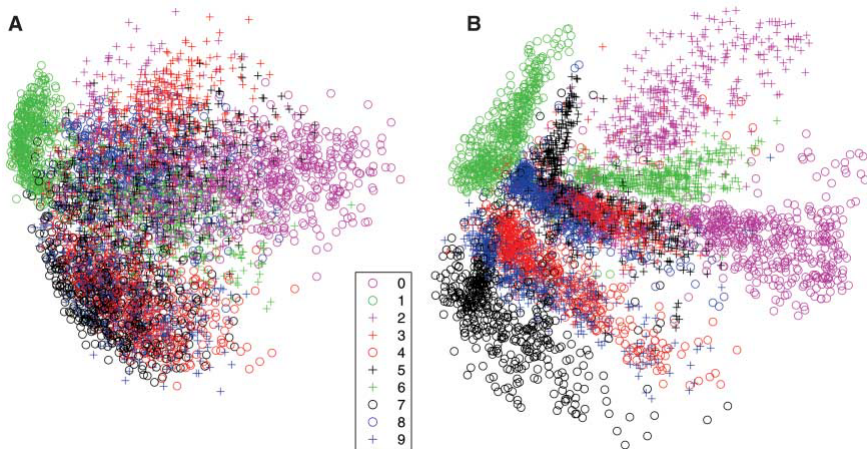


Figure: PCA vs Autoencoder for MNIST digits.

Undercomplete autoencoders

Classification application

- Labeled data is scarce and small amounts of data can lead to overfit.
- Huge amount of unlabeled data.

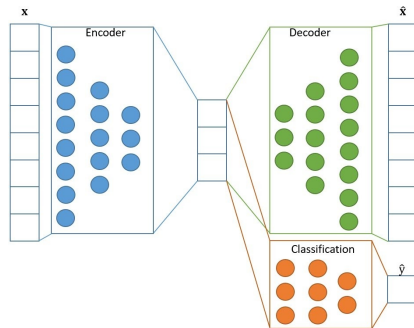


Figure: Example of the use of an autoencoder to solve classification tasks².

²Bank, D., Koenigstein, N., & Giryas, R. (2023). Autoencoders. Machine learning for data science handbook, 353-374.

Roadmap

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders**
- 4 Denoising autoencoders
- 5 Regularized autoencoders
- 6 Variational autoencoders

Convolutional autoencoders

- Replace dense layers with conv / (transposed) conv.
- Preserves locality; better image reconstructions.

Convolutional Autoencoders (CAE)³

- **Encoder:**
Conv $\rightarrow \sigma(h) \rightarrow$ Pool.
- **Code:**
small $H' \times W' \times C'$ feature tensor.
- **Decoder:**
Transposed Conv / UpSampling.
- **Why better for images?** Weight sharing & locality priors.
- Typical losses: ℓ_2 (pixel MSE), ℓ_1 , SSIM, or hybrid.

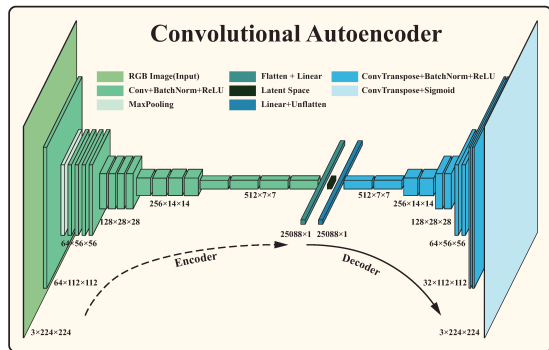


Figure: CAE: conv bottleneck preserves spatial structure

³Masci, Jonathan, et al. "Stacked convolutional auto-encoders for hierarchical feature extraction."

- **Linear interpolation:**

$$z_{\alpha} = (1 - \alpha)z_1 + \alpha z_2, \quad \alpha \in [0, 1]$$

- Decode $g_{\phi}(z_{\alpha})$ to visualize smooth morphs (e.g., dog $\rightarrow$ bird).

Interpolating

Image space



Interpolating

Latent space



Latent Space: Interpolation & Arithmetic

- **Linear interpolation:**

$$z_{\alpha} = (1 - \alpha)z_1 + \alpha z_2, \quad \alpha \in [0, 1]$$

- Decode $g_{\phi}(z_{\alpha})$ to visualize smooth morphs (e.g., dog $\rightarrow$ bird).
- **Attribute vectors:** $z_{\text{smile}} - z_{\text{neutral}}$; add to new z .



Figure: Smooth changes along semantic directions

Transformations in Latent Space

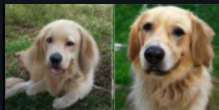


Fig. 5: Magnificación



Fig. 6: Cambio de posición



Fig. 7: Cambio de brillo

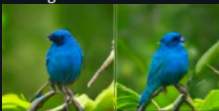


Fig. 8: Rotación (tenga en cuenta que la rotación podría ser 3D)

Roadmap

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders
- 4 Denoising autoencoders**
- 5 Regularized autoencoders
- 6 Variational autoencoders

- Denoising autoencoders

$$\min \mathcal{L}(x, g(f(\tilde{x}))) \quad (7)$$

where $\tilde{x}$ is a copy of x that has been corrupted by some noise. i.e.

$$p(\tilde{x}|x)$$

- Noise models: Gaussian, masking (salt & pepper), dropout-like corruption.

Denoising autoencoder

Superesolution example

- Task: To recover a high resolution image from a low resolution image.

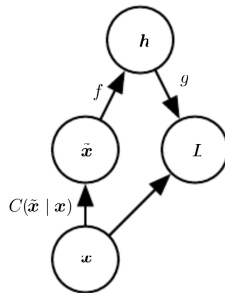
$$r = g(f(\tilde{x})) \quad (8)$$

where x is the original image, $\tilde{x}$ is the low resolution image and r the recovered image.



Denoising autoencoders

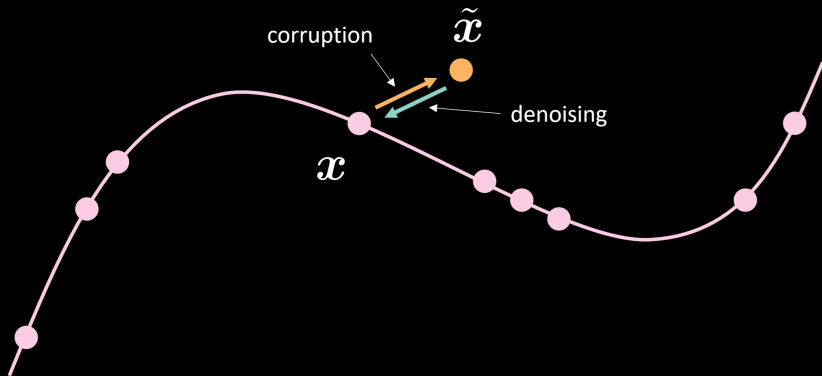
- Receives a corrupted data point as input and it is trained to predict the original.
- **Training:** sample $\tilde{x} \sim q(\tilde{x}|x)$; encode–decode; penalize $L(x, r)$.



- **Intuition:** forces encoder to learn *stable* features that survive corruption.

$$\tilde{x} = x + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2 I) \Rightarrow \min_{\theta, \phi} \mathbb{E}_{\tilde{x}} \|x - g(f(\tilde{x}))\|^2$$

$$\tilde{x} \sim p(\tilde{x} | x)$$



Regularized autoencoders

Notation and organization

- Regularized autoencoders try to avoid the identity function inside the autoencoder.

$$\begin{pmatrix} h_1 \\ h_2 \\ h_3 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \quad (9)$$

- They add a term to the loss function:

$$\mathcal{L}(x, r) = E(x, r) + \Omega(c) \quad (10)$$

where $E(x, r)$ is the standard reconstruction error and $\Omega(c)$ is a regularization term.

Roadmap

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders
- 4 Denoising autoencoders
- 5 Regularized autoencoders**
- 6 Variational autoencoders

Regularized autoencoders

Sparse Autoencoders

- Sparse Autoencoders encourage near-zero activations in the code layer h .

$$\min \mathcal{L}(x, r) = E(x, r) + \Omega(h) \quad (11)$$

- $\Omega(h)$ is a sparsity penalty on the code layer h . This penalty enforces that only a small number of hidden units are active for any given input.

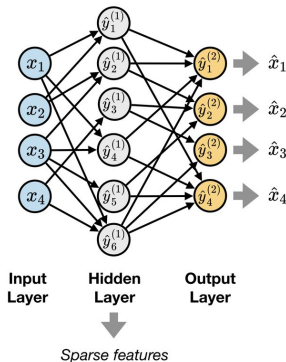


Figure: Example of a sparse autoencoder^a.

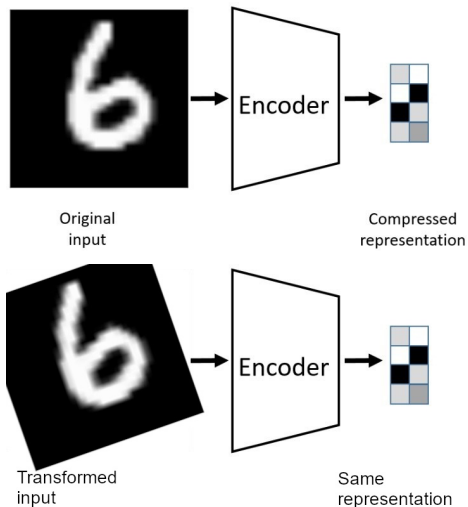
^aDutta, S., Bai, Z., Jeong, H., Low, T. M., & Grover, P.

Regularized autoencoders

Contractive Autoencoders

- They are designed to learn a representation (**$\mathbf{h}$**) that is **robust to small variations** in the input data (**$\mathbf{x}$**).

$$\tilde{x} = T(x) \quad (12)$$



Regularized autoencoders

Contractive Autoencoders

- The learned features are forced to be **locally invariant** to input perturbations.
- The overall objective function to minimize is:

$$\mathcal{L}(x, g(f(\tilde{\mathbf{x}}))) = E(x, g(f(\tilde{\mathbf{x}}))) + \Omega(x)$$

- Regularization Term, $\Omega(x)$: The penalty term is the squared Frobenius norm of the Jacobian matrix of the encoder's output with respect to the input:

$$\Omega(x) = \lambda \left\| \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2$$

where $f(\mathbf{x})$ is the encoder function, λ is the regularization coefficient, $\Omega(x)$ forces the partial derivatives to be small, thus "contracting" the input manifold.

Contractive Autoencoders

Applications

- **Robust Feature Extraction:** Scenarios where the input data contains noise or slight variations (e.g., small shifts or rotations in images), as the model is penalized for changing its embedded when the input changes slightly.
- **Learning Stable Features:** By enforcing local invariance, CAEs learn features that are stable, making them valuable for tasks like transfer learning.
- **Improving Deep Network Performance:** The features learned by a CAE can be used to initialize the weights of deeper supervised networks, providing a robust starting point that often leads to faster convergence and better final performance compared to random initialization.

Roadmap

- 1 Motivation
- 2 Autoencoders
- 3 Convolutional autoencoders
- 4 Denoising autoencoders
- 5 Regularized autoencoders
- 6 Variational autoencoders**

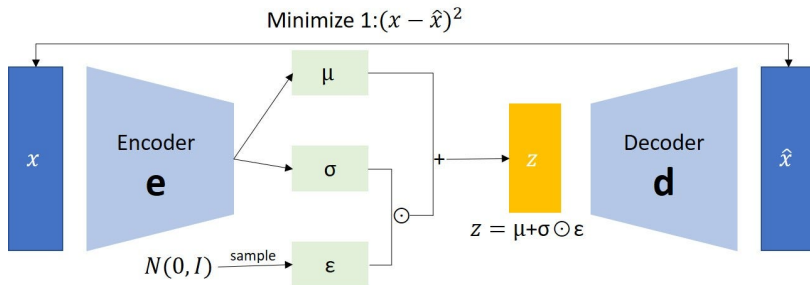
Variational Autoencoders (VAE)

From Deterministic Codes to Probabilistic Latents

- **Latent variable model:** $z \sim p(z) = \mathcal{N}(0, I)$, $x \sim p_\phi(x|z)$.
- **Encoder (inference):** $q_\theta(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x)))$.
- **Reparameterization:** $z = \mu(x) + \sigma(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.

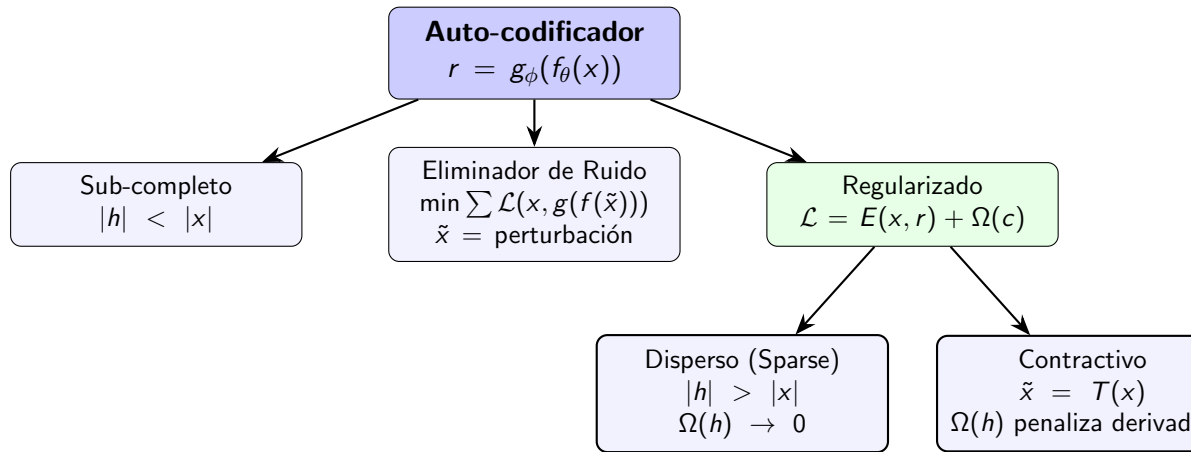
$$\mathcal{L}_{\text{VAE}}(x) = \underbrace{\mathbb{E}_{q_\theta(z|x)}[-\log p_\phi(x|z)]}_{\text{reconstruction}} + \beta \underbrace{D_{KL}(q_\theta(z|x) \parallel p(z))}_{\text{regularization}}$$

- **β -VAE:** increase β for more disentangled factors (trade-off with recon).



Minimize 2: $\frac{1}{2} \sum_{i=1}^N (\exp(\sigma_i) - (1 + \sigma_i) + \mu_i^2)$

Figure: Variational autoencoder illustration



- Normalize inputs (e.g., to $[0, 1]$ or $[-1, 1]$); monitor loss curves.
- Watch batch size vs. learning rate; too large LR can cause softmax/exp overflow.
- For images, prefer CAE or VAE with conv backbones for sharper reconstructions.
- Use validation recon error and sample quality (and SSIM/LPIPS) to compare models.

- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. Deep learning. MIT press, 2016.
- Bank, Dor, Noam Koenigstein, and Raja Giryes. "Autoencoders. arxiv." arXiv preprint arXiv:2003.05991 (2020): 2593-2613.
- NYU, Aprendizaje profundo,
<https://atcold.github.io/pytorch-Deep-Learning/es/week07/07-3/>